

# **Experiential Learning Project Assessment-I**

on

## **“Address Book in Python”**

**under the course | Python Programming Language (N-PCCCM304P)**

**Presented By  
BRIJESH  
YOGESH  
THAKUR**



**Under the Guidance of  
Mrs. Mayuri Getme**

**Session: 2026–27 (ODD) | Department of CSE (AI&ML)**

# Contents

- Problem statement
- Objectives
- Algorithm
- Flowchart / Pseudocode
- Understanding of Python logic
- GitHub: account, repository creation, code upload, meaningful commits, collaboration and individual contribution
- Application features: add, view, search, update and delete contacts
- Permanent storage using file handling or a database

# 1. Problem Statement

## The Need

Managing personal and professional contacts manually can become difficult as the number of contacts grows.

The project develops a Python-based Address Book that stores contact details and provides simple operations to manage them.

## Required Information

- Name
- Phone number
- Email address
- Postal address
- Other relevant contact information

Records should be stored permanently using file handling or a database.

## 2. Objectives

- Develop a Python-based Address Book application.
- Store and organize personal and professional contact information efficiently.
- Provide an interactive menu-driven interface.
- Allow users to add, view, search, update and delete contacts.
- Maintain useful details such as name, phone number, email and postal address.
- Ensure contact records are stored permanently using file handling or a database.

### 3. Main Features

#### **ADD CONTACT**

Enter and save a new contact with required details.

#### **VIEW CONTACTS**

Display all saved contacts in an organized form.

#### **SEARCH**

Find a contact quickly using a suitable detail such as name or phone.

#### **UPDATE**

Edit existing contact information when details change.

#### **DELETE**

Remove an unwanted contact from the address book.

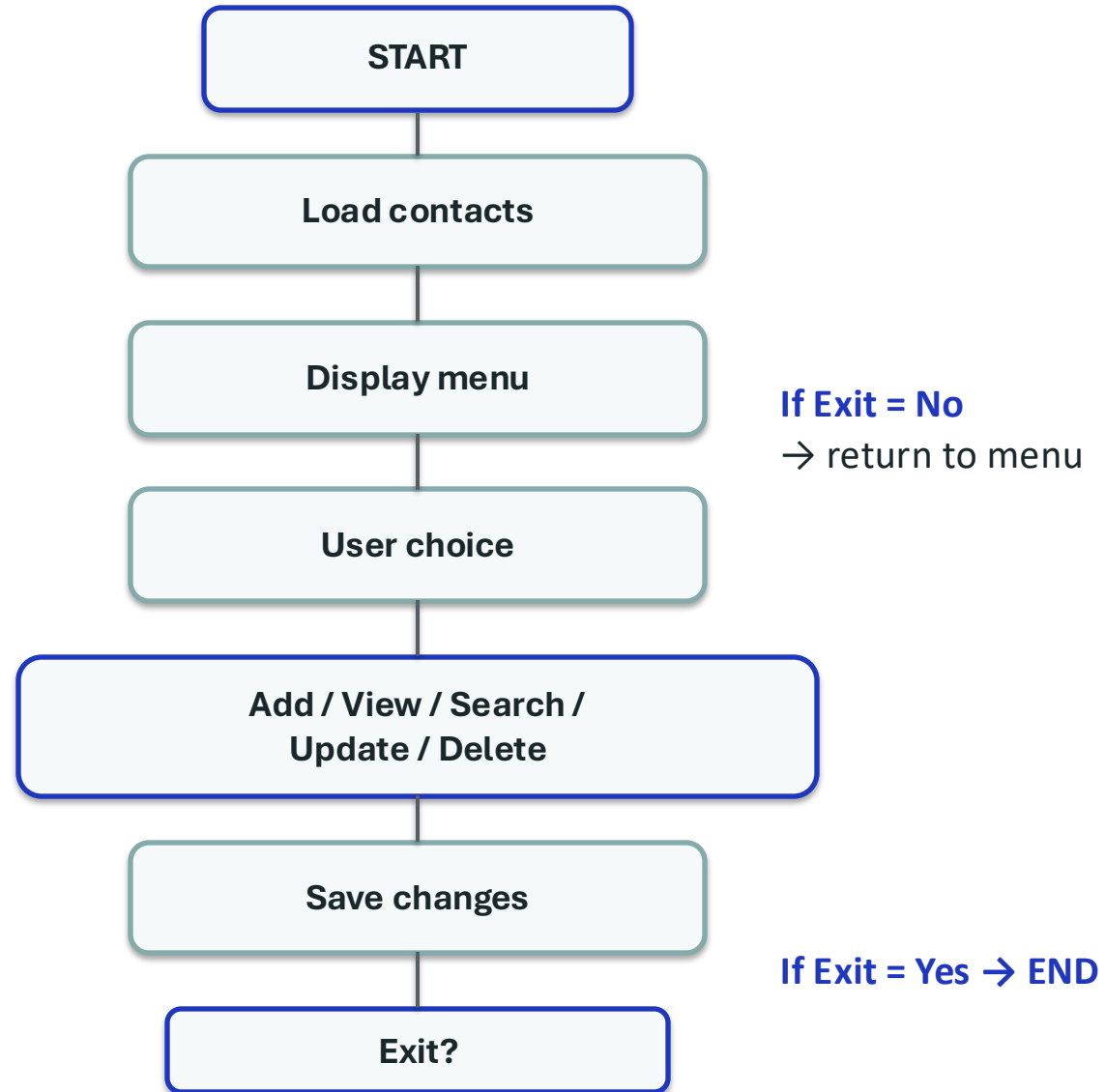
#### **PERMANENT STORAGE**

Keep records available after the program is closed.

## 4. Algorithm

- Start the Address Book application.
- Load existing contacts from the file/database.
- Display the menu: Add, View, Search, Update, Delete, Exit.
- Accept the user's choice.
- Perform the selected operation on contact records.
- Save changes permanently after add/update/delete operations.
- Repeat the menu until the user selects Exit.
- End the program.

## 5. Flowchart / Program Logic



## 6. Pseudocode

```
BEGIN
  Load contacts
  REPEAT
    Display menu
    Read choice

    IF choice = Add
      Enter and store contact
    ELSE IF choice = View
      Display contacts
    ELSE IF choice = Search
      Find matching contact
    ELSE IF choice = Update
      Modify selected contact
    ELSE IF choice = Delete
      Remove selected contact
    ELSE IF choice = Exit
      Stop
    END IF
  UNTIL choice = Exit
END IF
```

Save updated records

UNTIL choice = Exit

# 7. Understanding of Python Logic

## Variables & Data

Store fields such as name, phone, email and address. A contact can be represented using a dictionary or a structured record.

## Functions

Separate operations into functions such as `add_contact()`, `view_contacts()`, `search_contact()`, `update_contact()` and `delete_contact()`.

## Conditions & Loops

`if/elif` handles menu choices, while loops keep the menu running until Exit is selected.

## File Handling

Use a file to save contact records permanently. Read records when the application starts and write updated records after changes.

## Error Handling

Validate user input and handle cases such as an empty address book, contact not found, or invalid menu choice.

## 8. Menu-Driven Interface

```
===== ADDRESS BOOK =====
```

1. Add Contact
2. View Contacts
3. Search Contact
4. Update Contact
5. Delete Contact
6. Exit

Enter your choice:

### Why Menu-Driven?

It gives the user a clear and interactive way to select operations without needing to know the internal program logic.

### User Flow

Choose an operation → enter or select contact details → perform the operation → save the updated records → return to menu.

## 9. GitHub Workflow & Collaboration

- Create or use a GitHub account for the project.
- Create a meaningful repository for the Address Book application.
- Upload the Python source code and supporting files.
- Use meaningful commits to record each major improvement.
- Collaborate by sharing the repository and coordinating changes.
- Show individual contribution clearly through commits and work completed.

### Example Commit

“Added search and update contact functions”

# 10. Expected Result & Conclusion

## Expected Result

A simple Python-based Address Book that can:

- Store contact information
- Organize and display records
- Search contacts
- Update existing details
- Delete contacts
- Preserve records permanently
- Provide an easy menu-driven experience

## Conclusion

The project applies Python programming logic to a practical real-life problem. It combines functions, conditions, loops, data handling and permanent storage in one useful application.

It also demonstrates project workflow through GitHub, meaningful commits, collaboration and individual contribution.

# THANK YOU

## Address Book in Python

